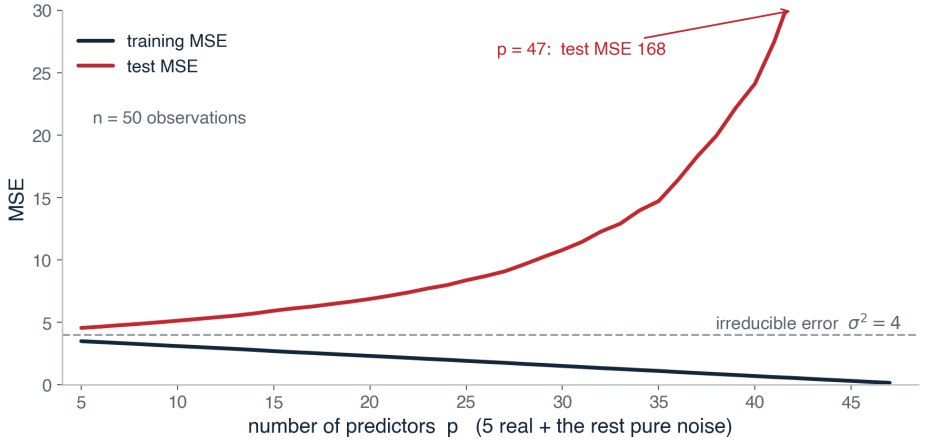
Ridge, Lasso and glmnet

MATH 4432 Statistical Machine Learning · Tutorial T10

Department of Mathematics, HKUST

Fall 2026

Too many predictors



least squares fits the noise · $p > n$: no unique fit at all

Two ways to fix it

subset selection

ISL §6.1

- keep some predictors, drop the rest
- each coefficient is *in* or *out*
- many models to search

shrinkage

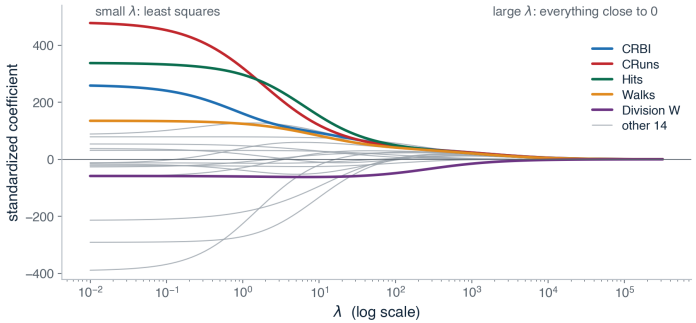
ISL §6.2 · today

- keep every predictor
- pull the coefficients towards 0
- one dial, λ, sets how hard

both accept a little **bias** to remove a lot of **variance** (T02)

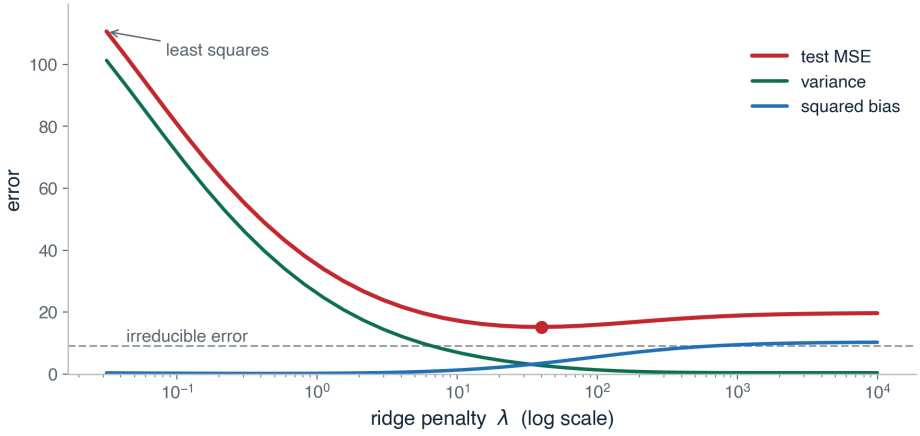
Ridge regression

$$\hat{\beta}_\lambda^R = \arg\min_\beta \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j \right)^2 + \lambda \sum_{j=1}^{p} \beta_j^2$$



Hitters: salaries of 263 players, 19 standardized predictors · β_0 is not penalized

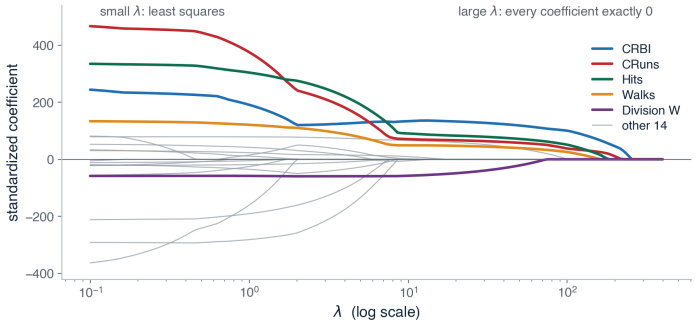
Shrinking trades bias for variance



simulation, $n = 50$, $p = 45$ · least squares: test MSE 111 · best ridge: 15

The lasso

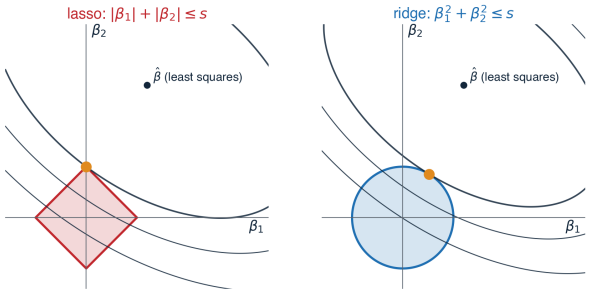
$$\hat{\beta}_\lambda^L = \arg\min_\beta \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j\right)^2 + \lambda \sum_{j=1}^{p} |\beta_j|$$



coefficients hit **exactly 0** one by one: the lasso **selects** variables

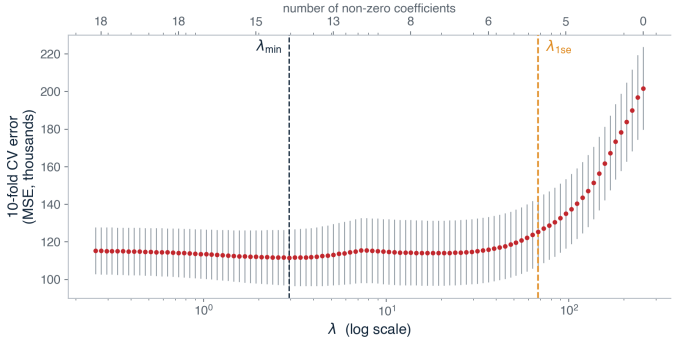
Why the lasso gives exact zeros

$$\min_{\beta} \text{RSS} \quad \text{subject to} \quad \sum_j |\beta_j| \le s \quad \text{or} \quad \sum_j \beta_j^2 \le s$$



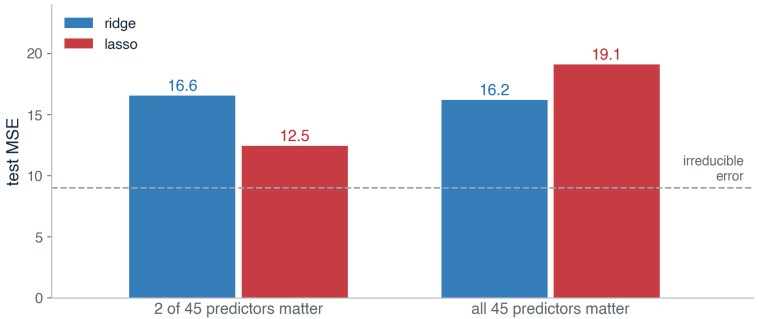
the RSS contours often touch the diamond first at a corner, where $\beta_1 = 0$

Choosing λ by cross-validation



- λ_{min}: smallest CV error · 13 predictors
- λ_{1se}: largest λ within one SE of the minimum · 6 predictors (T07)

Ridge or lasso?



simulation, $n = 50$, $p = 45$, λ by 10-fold CV · least squares: 125 and 147

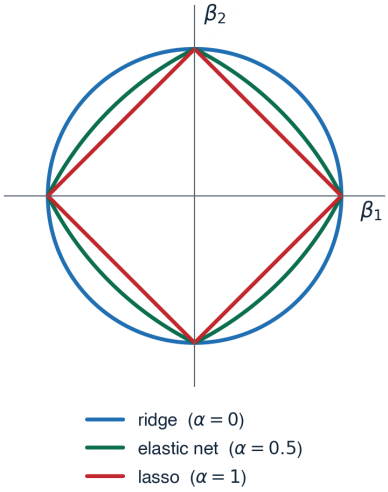
few strong signals: lasso · many small effects: ridge · unknown: let CV decide

The elastic net

$$\text{RSS} + \lambda\left[\frac{1-\alpha}{2}\sum_j \beta_j^2 + \alpha\sum_j |\beta_j|\right]$$

- $\alpha = 1$: lasso $\alpha = 0$: ridge
- in between: zeros like the lasso
- correlated predictors enter together, like ridge

  the lasso tends to keep one of them



glmnet: one package, the whole family

$$\min_{\beta_0,\beta} \frac{1}{n}\sum_{i=1}^{n} \ell\left(y_i,\ \beta_0 + x_i^\top \beta\right) \ +\ \lambda\left[\ \tfrac{1-\alpha}{2}\ \|\beta\|_2^2 + \alpha\ \|\beta\|_1\right]$$

`family`	response
`"gaussian"`	a number (salary)
`"binomial"`	two classes
`"multinomial"`	K classes (tumour type)
`"poisson"`	counts
`"cox"`	survival times

- ℓ: squared error/2, or –log-likelihood
- a path of 100 λ's in one call
- standardizes the predictors for you
- R: Friedman, Hastie & Tibshirani

glmnet in R, the same in Python

R · glmnet

```r
library(glmnet)
x <- model.matrix(Salary ~ ., Hitters)[,-1]
fit <- glmnet(x, y, alpha = 1)  # path
cv <- cv.glmnet(x, y)           # 10-fold
coef(cv, s = "lambda.1se")
glmnet(x, y, alpha = 0.5)       # e-net
glmnet(x, k, family = "multinomial")
```

Python · scikit-learn

```python
Xs = StandardScaler().fit_transform(X)
lasso_path(Xs, y)               # path
LassoCV(cv=10).fit(Xs, y)       # 10-fold
# lambda.1se: by hand, next slide
ElasticNet(alpha=lam, l1_ratio=0.5)
LogisticRegression(C=1/(n*lam),
    l1_ratio=1, solver="saga")
```

- names swapped: glmnet `alpha` = `l1_ratio`, glmnet `lambda` = `alpha` (logistic: `C` = $1/n\lambda$)
- scikit-learn does not standardize: add `StandardScaler`
- lasso: same λ, same coefficients (checked) · ridge: λ scaled differently

1 · the lasso with cross-validation

```
X = pd.get_dummies(hitters.drop(columns="Salary"), drop_first=True, dtype=float)
y = hitters["Salary"]
model = make_pipeline(StandardScaler(),
                      LassoCV(cv=KFold(10, shuffle=True, random_state=0)))
cv = model.fit(X, y)[-1]
m  = cv.mse_path_.mean(axis=1)                          # CV error for each lambda
se = cv.mse_path_.std(axis=1, ddof=1) / np.sqrt(10)   # and its standard error
lam_1se = cv.alphas_[m <= m.min() + se[m.argmin()]].max()
fit_1se = make_pipeline(StandardScaler(), Lasso(alpha=lam_1se)).fit(X, y)
print(cv.alpha_.round(2), (cv.coef_ != 0).sum())
print(lam_1se.round(1), X.columns[fit_1se[-1].coef_ != 0].tolist())
```

```
2.94 13
67.8 ['Hits', 'Walks', 'CRuns', 'CRBI', 'PutOuts', 'Division_W']
```

glmnet's `lambda.min` and `lambda.1se` · 13 or 6 of the 19 predictors

2 · least squares, ridge, lasso, elastic net

```
models = {"least squares": LinearRegression(),
          "ridge": RidgeCV(np.logspace(-2, 4, 100), cv=10),
          "lasso": LassoCV(cv=10),
          "elastic net": ElasticNetCV(l1_ratio=0.5, cv=10)}
res = []
for seed in range(20):                # 20 random half splits
    X_tr, X_te, y_tr, y_te = train_test_split(
        X, y, test_size=0.5, random_state=seed)
    for name, m in models.items():
        fit = make_pipeline(StandardScaler(), m)
        fit.fit(X_tr, y_tr)
        mse = mean_squared_error(y_te, fit.predict(X_te))
        res.append({"model": name, "MSE": mse / 1000,
                    "p": (fit[-1].coef_ != 0).sum()})
res = pd.DataFrame(res).groupby("model", sort=False).mean()
print(res.round(1))
```

model	MSE	p
least squares	137.6	19.0
ridge	123.0	19.0
lasso	130.9	11.5
elastic net	122.2	18.8

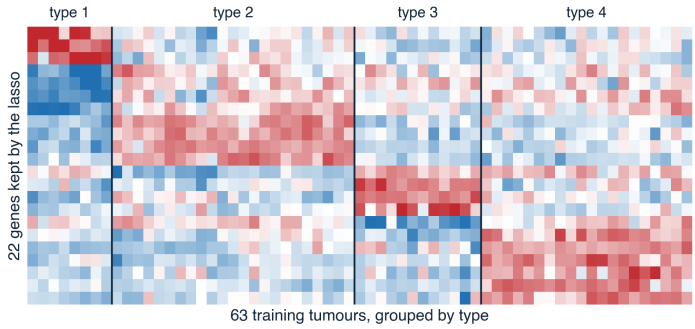
test MSE, in thousands

p: predictors used

mean of 20 splits

predicting the mean: 199.4

Application: which tumour is it?



- 2308 genes, 63 training and 20 test tumours, 4 types (Khan et al., 2001)
- $p \gg n$: least squares has no unique answer
- multinomial lasso: 22 genes $\Rightarrow$ all 20 test tumours correct

3 · the multinomial lasso

```
n, lams = len(y_tr), np.logspace(-0.5, -2, 10)          # 63 tumours; lambda 0.32 ... 0.01
model = make_pipeline(StandardScaler(),
                      LogisticRegression(l1_ratio=1, solver="saga", tol=1e-3,
                                         max_iter=200000, random_state=0))
search = GridSearchCV(model, {"logisticregression__C": 1 / (n * lams)},
                      cv=5, scoring="accuracy").fit(X_tr, y_tr)
best = search.best_estimator_.set_params(logisticregression__tol=1e-6)   # refit, tighter
best.fit(X_tr, y_tr)
lam = 1 / (n * search.best_params_["logisticregression__C"])
genes = (best[-1].coef_ != 0).any(axis=0)               # used by any of the 4 types
print(lam.round(3), genes.sum(), (best.predict(X_te) == y_te).mean())
```

```
0.1 22 1.0
```

λ · genes used · test accuracy

R's `cv.glmnet`: λ = 0.065, 24 genes, 20/20, in under a second

Take away

- many predictors: least squares overfits; shrinking trades a little bias for much less variance
- ridge ($\sum \beta_j^2$) shrinks every coefficient; the lasso ($\sum |\beta_j|$) sets some exactly to 0
- standardize first, then choose λ by cross-validation (λ_{min} or λ_{1se})
- neither always wins; the elastic net mixes the two
- glmnet: one recipe for numbers, classes, counts and survival, even when $p \gg n$

References

- James, Witten, Hastie & Tibshirani, *An Introduction to Statistical Learning*, §6.2, lab §6.5.2
- Hastie, Tibshirani & Friedman, *The Elements of Statistical Learning*, §3.4, §18.4
- Hoerl & Kennard (1970), Ridge regression · Tibshirani (1996), the lasso · Zou & Hastie (2005), the elastic net
- Friedman, Hastie & Tibshirani (2010), Regularization paths for generalized linear models via coordinate descent, *J. Statistical Software*
- Khan et al. (2001), *Nature Medicine* · the tumour data
- `T10_Ridge_Lasso.ipynb` — every number on these slides · `T10_glmnet.R` — the same in R